CYBERSECURITY MAJOR PROJECT REPORT

Full-Chain Enterprise Perimeter Breach & Host Takeover Audit

Assigned Group: Group 5
Primary Operator: nikita456 (nikita456@AstraNode)
TryHackMe Lab: Ignite Room
Target Enterprise: AstraNode
Target IP Address: 10.48.146.129
Lab Profile: tryhackme.com/p/nikita456

1. EXECUTIVE SUMMARY

During a controlled security evaluation of the **AstraNode** infrastructure, Group 5 executed a comprehensive, multi-stage cyber threat simulation. The primary objective was to audit the target's external web attack surface, evaluate input validation mechanisms, establish persistent session management, and test local operating system security controls.

The assessment revealed critical security deficiencies, including unauthenticated web application exposure (Apache 2.4.18 running FUEL CMS), insufficient server-side file upload validation allowing arbitrary web shell execution, and misconfigured binary permissions (SUID bits) enabling immediate root-level privilege escalation.

2. CYBER KILL CHAIN & EXECUTION SUMMARY

Kill Chain Phase	Technical Vector & Command	Impact / Evidence Captured	Severity
1. Reconnaissance	`nmap -sV -sC -p- -T4 -oN recon_results.txt`	Identified HTTP Port 80 (FUEL CMS)	INFORMATIONAL
2. Web Exploitation	`ffuf -u http://10.48.146.129/FUZZ -w ...`	Discovered hidden endpoints & assets	HIGH
3. Upload Fuzzing	`ffuf -request upload_req.txt ...`	Identified upload validation gaps	HIGH
4. C2 Session	`msfconsole` + `exploit/multi/handler`	Established Meterpreter reverse TCP connection	CRITICAL
5. Local PrivEsc	`find / -perm -4000 -type f 2>/dev/null`	Discovered SUID binaries for root takeover	CRITICAL

Audit Record & Proof-of-Concept (PoC) Note
All terminal outputs, commands, and screenshots contained in this report were captured directly from operator workstation `nikita456@AstraNode` during the active security assessment of the designated target environment.

3. ATTACK LIFECYCLE & TERMINAL VERIFICATION

Phase 1: OSINT & Active Reconnaissance

Network port scanning was conducted using `nmap` against target `10.48.146.129`. The scan identified an open HTTP service on port 80 running Apache httpd 2.4.18 (Ubuntu) hosting FUEL CMS, alongside an entry in `robots.txt` disclosing the `/fuel/` admin directory.



Figure 1: Active Nmap service scan executed on operator workstation (nikita456@AstraNode).

Phase 2: Web Endpoint Fuzzing & Directory Enumeration

Directory fuzzing was executed with `ffuf` utilizing standard wordlists to uncover accessible web subdirectories, confirming endpoints such as `/assets`, `index.php`, and `/home`.



Figure 2: FFUF directory fuzzing execution against http://10.48.146.129/.

Phase 3: Web Shell Upload & Request Fuzzing

To test the web application's upload handlers, parameter and extension fuzzing was performed using `ffuf` with web content extension wordlists to locate unvalidated file upload vectors.



Figure 3: FFUF extension and upload request analysis on target upload endpoints.

Phase 4: Metasploit Integration & C2 Handler Setup

To convert initial command access into an interactive session, Metasploit Framework's `exploit/multi/handler` module was configured on operator IP `192.168.133.185` listening on port 4444 with payload php`/meterpreter/reverse_tcp`.



Figure 4: Metasploit Framework multi/handler reverse listener initialization.

Phase 5: Linux Local Privilege Escalation (SUID Enumeration)

Following initial host access, system permissions were audited using Linux SUID binary searching. Executing `find / -perm -4000 -type f 2>/dev/null` uncovered system binaries with elevated execution privileges suitable for privilege escalation to `root`.



Figure 5: SUID permission search output listing system binaries on target host.

4. DEFENSIVE REMEDIATION & HARDENING PLAN

Control 1: Secure Application Code (PHP File Upload Hardening)

Backend Validation Script (/var/www/html/upload.php)

```
<?php // Enforce strict MIME-type and extension validation $allowed_mimes = ['image/jpeg', 'image/png']; $finfo = finfo_open(FILEINFO_MIME_TYPE); $mime = finfo_file($finfo, $_FILES['uploadedfile']['tmp_name']); if (!in_array($mime, $allowed_mimes)) { die("Security Alert: Unauthorized file type detected. Upload blocked."); } ?>
```

Control 2: Web Server Security Configuration (Apache Execution Suppression)

Apache Directory Hardening (/etc/apache2/sites-available/000-default.conf)

```
<Directory /var/www/html/uploads> php_admin_value engine Off Options -ExecCGI AllowOverride None </Directory>
```